You should establish a follow-up procedure to ensure that items on the issues list have been properly corrected. Unless this is done, it is possible that issues raised can "fall between the cracks." One approach is to assign the responsibility for follow-up to the review leader.

16.6.3 Review Guidelines

Guidelines for conducting formal technical reviews must be established in advance, distributed to all reviewers, agreed upon, and then followed. A review that is uncontrolled can often be worse than no review at all. The following represents a minimum set of guidelines for formal technical reviews:

1. **Review the product, not the producer.** An FTR involves people and egos. Conducted properly, the FTR should leave all participants with a warm feeling of accomplishment. Conducted improperly, the FTR can take on the aura of an inquisition. Errors should be pointed out gently; the tone of the meeting should be loose and constructive; the intent should not be to embarrass or belittle. The review leader should conduct the review meeting to ensure that the proper tone and attitude are maintained and should immediately halt a review that has gotten out of control.
2. **Set an agenda and maintain it.** One of the key maladies of meetings of all types is drift. An FTR must be kept on track and on schedule. The review leader is chartered with the responsibility for maintaining the meeting schedule and should not be afraid to nudge people when drift sets in.
3. **Limit debate and rebuttal.** When an issue is raised by a reviewer, there may not be universal agreement on its impact. Rather than spending time debating the question, the issue should be recorded for further discussion off-line.
4. **Enunciate problem areas, but don't attempt to solve every problem noted.** A review is not a problem-solving session. The solution of a problem can often be accomplished by the producer alone or with the help of only one other individual. Problem solving should be postponed until after the review meeting.
5. **Take written notes.** It is sometimes a good idea for the recorder to make notes on a wall board, so that wording and priorities can be assessed by other reviewers as information is recorded.
6. **Limit the number of participants and insist upon advance preparation.** Two heads are better than one, but 14 are not necessarily better than 4. Keep the number of people involved to the necessary minimum. However, all review team members must prepare in advance.
7. **Develop a checklist for each product that is likely to be reviewed.** A checklist helps the review leader to structure the FTR meeting and helps each reviewer to focus on important issues. Checklists should be developed for analysis, design, code, and even testing work products.
8. **Allocate resources and schedule time for FTRs.** For reviews to be effective, they should be scheduled as a task during the software process. In addition, time should be scheduled for the inevitable modifications that will occur as the result of an FTR.

9. **Conduct meaningful training for all reviewers.** The training should stress both process-related issues and the human psychological side of reviews.
10. **Review your early reviews.** Debriefing can be beneficial in uncovering problems with the review process itself. The very first product to be reviewed should be the review guidelines themselves.

Because many variables (e.g., number of participants, type of work products, timing and length, specific review approach) have an impact on a successful review, a software organization should experiment to determine what approach works best in a local context.

In an ideal setting, every software-engineering work product would undergo a formal technical review. In the real world of software projects, resources are limited and time is short. As a consequence, reviews are often skipped, even though their value as a quality control mechanism is recognized. However, full FTR resources should be used on those work products that are likely to be error prone.

SAFEHOME



Quality Issues

The scene: Doug Miller's office as the *SafeHome* software project begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and other members of the product software engineering team.

The conversation:

Doug: I know we didn't spend time developing a quality plan for this project, but we're already into it and we have to consider quality . . . right?

Jamie: Sure. We've already decided that as we develop the requirements model [Chapter 8], Ed has committed to develop a testing procedure for each requirement.

Doug: That's really good, but we're not going to wait until testing to evaluate quality, are we?

Vinod: No! Of course not. We've got reviews scheduled into the project plan for this software increment. We'll begin quality control with the reviews.

Jamie: I'm a bit concerned that we won't have enough time to conduct all the reviews. In fact, I know we won't.

Doug: Hmmm. So what do you propose?

Jamie: I say we select those elements of the requirements and design model that are most critical to *SafeHome* and review them.

Vinod: But what if we miss something in a part of the model we don't review?

Jamie: Maybe . . . but I'm not sure we even have time to review every element of the models.

Vinod: What do you want us to do, Doug?

Doug: Let's steal something from Extreme Programming [Chapter 3]. We'll develop the elements of each model in pairs—two people—and conduct an informal review of each as we go. We'll then target "critical" elements for a more formal team review, but keep those reviews to a minimum. That way, everything gets looked at by more than one set of eyes, but we still maintain our delivery dates.

Jamie: That means we're going to have to revise the schedule.

Doug: So be it. Quality trumps schedule on this project.

16.7 Postmortem Evaluations

Many lessons can be learned if a software team takes the time to evaluate the results of a software project after the software has been delivered to end users. Baaz and his colleagues [Baa10] suggest the use of a *postmortem evaluation* (PME) as a mechanism to determine what went right and what went wrong when software engineering process and practice is applied in a specific project.

Unlike an FTR that focuses on a specific work product, it is more like a Scrum retrospective (Section 3.4.5). A PME examines the entire software project, focusing on both "*excellences* (that is, achievements and positive experiences) and *challenges* (problems and negative experiences)" [Baa10]. Often conducted in a workshop format, a PME is attended by members of the software team and stakeholders. The intent is to identify excellences and challenges and to extract lessons learned from both. The objective is to suggest improvements to both process and practice going forward. Many software engineers regard PME documents as some of the most valuable documents to save in the project archive.

16.8 Agile Reviews

It's not surprising that some software engineers balk at the thought of including reviews of any kind in agile development processes. Yet failing to catch defects early can be costly in terms of time and resources. Ignoring technical debt does not make it go away. Agile developers have the same need to find defects early (and often) that all software developers have. Admittedly, it may be somewhat more difficult to get agile developers to make use of metrics, but many of them can be collected unobtrusively.

If we take a closer look at the Scrum framework (Section 3.4), there are several places where informal and formal reviews take place. During the sprint planning meeting, user stories are reviewed and ordered according to priority, before selecting the user stories to include in the next sprint. The daily Scrum meeting is an informal way to ensure that the team members are all working on the same priorities and to catch any defects that may prevent completing the sprint on time. Agile developers often use pair programming, another informal review technique. The sprint review meeting is often conducted using guidelines similar to those discussed for a formal technical review. The code producers walk through the user stories selected for the sprint and demonstrate to the product owner that all functionality is present. Unlike the FTR, the product owner has the final word on whether to accept the sprint prototype or not.

If we look at the evaluate prototype portion of our recommended process model (Section 4.5), this task is also likely to be conducted as a formal technical review with development risk assessment added to it. We mentioned previously (Section 16.7) that the sprint retrospective meeting is really similar to the project postmortem meeting in that the development team is trying to capture its lessons learned. A major aspect of software quality assurance is being able to repeat your successes and avoid repeating your mistakes.

16.9 SUMMARY

The intent of every technical review is to find errors and uncover issues that would have a negative impact on the software to be deployed. The sooner an error is uncovered and corrected, the less likely that error will propagate to other software engineering work products and amplify itself, resulting in significantly more effort to correct it.

To determine whether quality control activities are working, a set of metrics should be collected. Review metrics focus on the effort required to conduct the review and the types and severity of errors uncovered during the review. Once metrics data are collected, they can be used to assess the efficacy of the reviews you do conduct. Industry data indicate that reviews provide a significant return on investment.

A reference model for review formality identifies the roles people play, planning and preparation, meeting structure, correction approach, and verification as the characteristics that indicate the degree of formality with which a review is conducted. Informal reviews are casual in nature but can still be used effectively to uncover errors. Formal reviews are more structured and have the highest probability of leading to high-quality software.

Informal reviews are characterized by minimal planning and preparation and little record keeping. Desk checks and pair programming fall into the informal review category.

A formal technical review is a stylized meeting that has been shown to be extremely effective in uncovering errors. Formal technical reviews establish defined roles for each reviewer, encourage planning and advance preparation, require the application of defined review guidelines, and mandate record keeping and status reporting.

PROBLEMS AND POINTS TO PONDER

16.1. Explain the difference between an *error* and a *defect.*

16.2. Why can't we just wait until testing to find and correct all software errors?

16.3. Assume that 10 errors have been introduced in the requirements model and that each error will be amplified by a factor of 2:1 into design and an additional 20 design errors are introduced and then amplified 1.5:1 into code where an additional 30 errors are introduced. Assume further that all unit testing will find 30 percent of all errors, integration will find 30 percent of the remaining errors, and validation tests will find 50 percent of the remaining errors. No reviews are conducted. How many errors will be released to the end users?

16.4. Reconsider the situation described in Problem 16.3, but now assume that requirements, design, and code reviews are conducted and are 60 percent effective in uncovering all errors at that step. How many errors will be released to the field?

16.5. Reconsider the situation described in Problems 16.3 and 16.4. If each of the errors released to the field costs $4800 to find and correct and each error found in review costs $240 to find and correct, how much money is saved by conducting reviews?

16.6. Describe the meaning of Figure 16.1 in your own words.

16.7. Can you think of a few instances in which a desk check might create problems rather than provide benefits?

16.8. A formal technical review is effective only if everyone has prepared in advance. How do you recognize a review participant who has not prepared? What do you do if you're the review leader?

16.9. How is technical debt addressed in agile process models?

16.10. Consider the review guidelines presented in Section 16.6.3. Which do you think is most important and why?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

17 SOFTWARE QUALITY ASSURANCE

The software engineering approach described in this book works toward a single goal: to produce on-time, high-quality software. Yet many readers will be challenged by the question: "What is software quality?"

KEY CONCEPTS

Bayesian inference 351
elements of software quality assurance 341
formal approaches 347
genetic algorithms 352
goals 346
ISO 9001:2015 standard 354
Six Sigma 349
software reliability 350
software safety 352
SQA plan 354
SQA tasks 343
statistical software quality assurance 347



QUICK LOOK

What is it? It's not enough to talk the talk by saying that software quality is important. You have to (1) explicitly define what is meant when you say "software quality," (2) create a set of activities that will help ensure that every software engineering work product exhibits high quality, (3) perform quality control and assurance activities on every software project, (4) use metrics to develop strategies for improving your software process and, as a consequence, the quality of the end product.

Who does it? Everyone involved in the software engineering process is responsible for quality.

Why is it important? You can do it right, or you can do it over again. If a software team stresses quality in all software engineering activities, it reduces the amount of rework that it must do. That results in lower costs, and more importantly, improved time to market.

What are the steps? Before software quality assurance (SQA) activities can be initiated, it is important to define *software quality* at several different levels of abstraction. Once you understand what quality is, a software team must identify a set of SQA activities that will filter errors out of work products before they are passed on.

What is the work product? A Software Quality Assurance Plan is created to define a software team's SQA strategy. During modeling and coding, the primary SQA work product is the output of technical reviews (Chapter 16). During testing (Chapters 19 through 21), test plans and procedures are produced. Other work products associated with process improvement may also be generated.

How do I ensure that I've done it right? Find errors before they become defects! That is, work to improve your defect removal efficiency (Chapter 23), thereby reducing the amount of rework that your software team must perform.